Cache Coherence Assignment

The objective of this assignment is to implement the Snooping based MSI and MESI cache coherence protocol in a simple cache coherence simulator. We have provided you with a cache simulator written in C++. The simulator outputs Cache and Bus statistics and also prints the contents of the cache after simulating memory requests coming from multiple cores.

- You have to fill in the implementation of two functions (handleProcRequest and handleBusRequest) in coherence.cpp file. Your coherence.cpp file should implement both MSI and MESI protocol in the same file. You can modify other files of the simulator for debugging purposes but your coherence.cpp file should work with the original simulator without any modifications.
- The evaluations for this assignment will be based on test cases. Two of the test cases will be public and can be used to check you code. You are encouraged to test your code with your own test cases.
- We will use a plagiarism checker on your code so make sure your code is original.
- Submit only the coherence.cpp file renamed as <ROLL_NO>.cpp (in capital letters). For example, if your roll number is CS19S026, then submit the coherence.cpp file renamed as CS19S026.cpp.

About the simulator

- The simulator implements a very simple model of a snooping based 4 core system. The Cache simulated is a 4 way set associative cache consisting of 4 sets. The cache block size is 64 Bytes and the cache uses LRU replacement policy. The cache paramters are specified in Cache.h.

- Running the simulator

```
# Compile using make
make
# This will create an executable called sim.

# Run the simulator using input file
./sim < input_file
# Or you can give inputs from standard input
./sim
# Followed by your input
```

Instructions

1. You are expected to use the functions declared in the header files in your coherence.cpp file.
2. Cache to Cache sharing of cache block is disabled. Cache flushes only happen when required (i.e. only when the block is dirty)
3. In the header files, if the argument is named as "address" then it expects the whole address to be provided. If the argument is named as "block_address" then it expects block address without the cache offset.
4. insertCacheBlock will return the block being evicted. When evicting dirty cache blocks send a Flush bus request (instead of a BusWB).
5. moveToMRU has to be used to move the accessed cache block to the MRU position. insertCacheBlock will put the block in the MRU position so a call to moveToMRU is not required after insertCacheBlock.
6. The bus will keep track of the counts of the bus requests. But for the cache you are expected to update the 6 variables (num_reads, num_read_misses, num_writes, num_write_misses, num_writebacks, num_invalidations).
    - num_reads : number of reads from the processor
    - num_read_misses : number of reads from the processor that missed in the cache
    - num_writes : number of writes from the processor
    - num_write_misses : number of writes from the processor that missed in the cache
    - num_writebacks : number of writebacks done to write a dirty block
    - num_invalidations : number of invalidations caused by the coherence protocol
7. In MESI protocol, to check if other caches have the cache block, first send a BusRd message and the other caches should set the shared line (using setSharedLine) in response to reading a BusRd request for a block that it has. And after sending the BusRd message, that first core can read the shared line using getSharedLine().

Input format

1. The first line of input specifies the protocol to simulate: MSI/MESI
2. The following lines will have memory references in the format: c rw 0xffff
    - c is the core number that is initiating the memory references
    - rw is either r or w indicating of the operation is a read or a write
    - 0xffff indicates the byte address being read or written.
3. The last line of the file has -1 to let the simulator know that there are no more operations.

Ouput format

The output of the simulator prints the Bus statistics (number of bus requests of each type). Then for each cache it prints the contents of the cache and cache statistics (misses, writebacks, invalidations).